RETAIL-DATA-AGENT · RESEARCH FOR STEP 5 (REPORTS) · BUILT 2026-11-01

One Report Question, Several SQL Queries

How the agent should answer report questions such as “create a Q1 report with action items for Q2”: send several SQL queries at once, run them at the same time, then write the report. Confirmed in a live test on Gemini and BigQuery (2026-10-31). Full research with every source: research/report-questions-db-agent.md. See also the architecture map.

In simple words

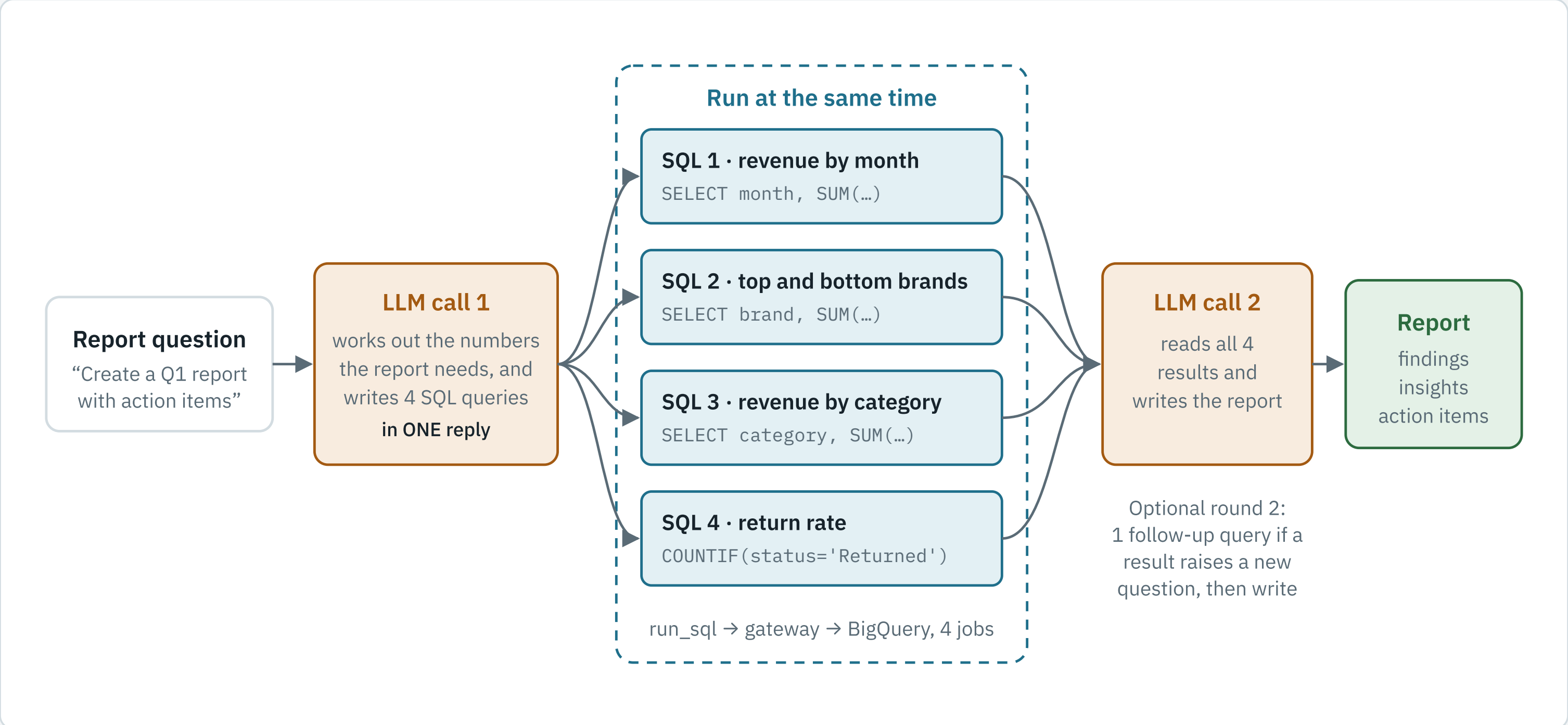
1. **One LLM reply can contain several SQL queries.** The model sends, for example, 4 `run_sql` calls in a single answer.[7]
2. **Our code runs them at the same time.** LangChain starts all 4 together, and each one becomes its own BigQuery job. They finish in about the time of the slowest one, not the sum.
3. **All 4 results go back to the model together**, and it writes the answer or report.

> That's 2 LLM calls for the whole report, however many queries the batch holds. If a result raises a new question, the model can send one more round of queries, which makes 3 calls. That's still inside the budget of 5 per question.

What got built (ticket 11, 2026-11-01)

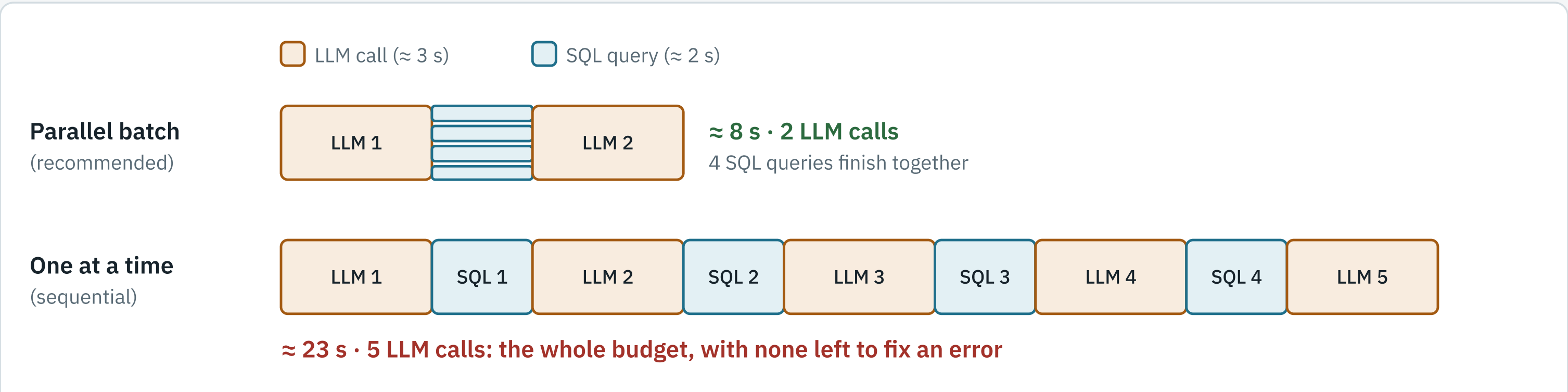
- **The batching lines are in `prompts/system.md`**: work out every number first, send independent queries together, at most 2 rounds, fix only a failed query.
- **The report is saved without an extra call.** The model writes it into a `save_report` tool, whose result is the final answer (`return_direct`). The code adds the owner, conversation, question and the SQL behind the numbers, and stores it in `data/reports.sqlite3`.
- **Live, 2026-11-01:** as the CEO on `gemini-3.6-flash`, “Create a report for Q1 2026 including insights and action items for Q2” took **2 LLM calls**: 6 queries in the first reply, run together in about 2.5 s, then the report written and saved.
- **The 5-call budget stays.** The open question about asking for more calls for reports is closed.

The flow


Report question
“Create a Q1 report with action items”
LLM call 1
works out the numbers the report needs, and writes 4 SQL queries in ONE reply
Run at the same time
SQL 1 · revenue by month
SELECT month, SUM(…)
SQL 2 · top and bottom brands
SELECT brand, SUM(…)
SQL 3 · revenue by category
SELECT category, SUM(…)
SQL 4 · return rate
COUNTIF(status='Returned')
run_sql → gateway → BigQuery, 4 jobs
LLM call 2
reads all 4 results and writes the report
Optional round 2: 1 follow-up query if a result raises a new question, then write
Report
findings
insights
action items

The model plans in its built-in thinking, so it doesn't write the plan as text first. Requiring text right before a tool call can cause a malformed-call error on Gemini 3.x.[7]

Why it's faster: the same 4 queries, two ways



Illustrative timing, using typical durations from the project log on 2026-10-30 (LLM calls 1.5–4.4 s, SQL 1.0–3.3 s). Every extra LLM call also sends the whole conversation to Gemini again, and the free tier limits input tokens per minute as well as requests.[4]

Compared: parallel, one at a time, or one big query

Example: a Q1 report that needs 4 independent numbers, with no errors. Token and time figures are estimates from the research note, not measurements.

	PARALLEL BATCH (RECOMMENDED)	ONE AT A TIME (SEQUENTIAL)	ONE BIG SQL QUERY (CTES)
How it works	One reply holds all 4 queries. They run together, and the next call writes the report.	Query, read the result, next query... then the report.	The model writes one long SQL statement that returns all 4 results.
LLM calls	**2** (3 with a follow-up round)	**5**: the whole budget, nothing left for a fix	**2**, but every fix rewrites the whole query
Time (4 queries)	≈ 8–12 s	≈ 20–25 s	≈ 8–12 s
Input tokens sent to Gemini	≈ 5.8k (conversation sent twice)	≈ 14.5k (sent 5 times)[4]	≈ 4k
BigQuery cost	≈ $0.0005 (≥ 80 MB billed: a 10 MB minimum per table, per query)[10]	Same as parallel	≈ $0.0002 (≥ 30 MB). All three are free within BigQuery's 1 TiB a month.[10]
If one query fails	Only that query is lost. The model fixes it in the next round (ticket 03 returns `Error: …`).	Only that step is lost	The whole report is lost
Accuracy	Good when the parts are independent. Planning calls up front matched or beat the step-by-step loop, by up to ~9%.[1][3] Gemini's built-in parallel calls get almost the same accuracy as a custom planner.[1]	Best when each step depends on the last result.[2] Known weak spots: repeating calls and stopping too early.[1]	Weakest. Long nested SQL is the hard case in text-to-SQL research.[5][6] BigQuery also runs a CTE once for each place it's used, so it doesn't save scanning.[11]
Best for	Reports, KPI summaries, “compare X and Y”	Drill-downs: “find the weakest category, then look into it”	One consistent table, e.g. state X vs state Y in one `GROUP BY`

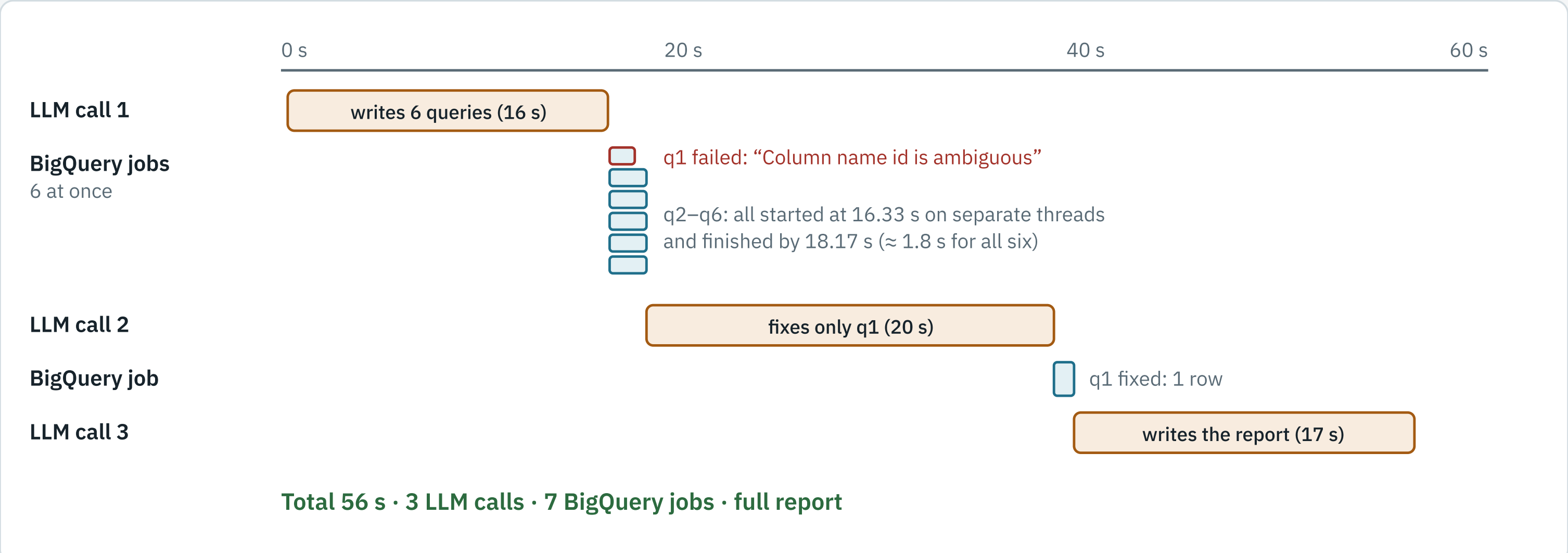
> Rule of thumb: send independent queries together, use a second round only when a query needs an earlier result, and compare groups inside one `GROUP BY` query. Google's agent guidance and Anthropic's tool docs give the same advice: prefer several specific calls, and batch only the calls that don't depend on each other.[8][9][12]

Live test on Gemini and BigQuery

Run on 2026-10-31 with the real agent code, a free Gemini key and real BigQuery. The question was “Create a report for Q1 2026 with insights and action items for Q2 2026.” “Batching lines” means the report instructions from the research note added to the system prompt.

MODEL	PROMPT	QUERIES PER REPLY	LLM CALLS	TIME	RESULT
gemini-3.6-flash	**with batching lines**	**6 in the first reply**	**3**	56 s	**✓ Full report**
gemini-3.6-flash	current	1, 1, 1, 1, 1	5 (limit)	62 s	**✕ No report: “I couldn't finish this question within 5 model**
gemini-3.5-flash-lite	current	1, 1, 1, 1	5	13 s	**✓ Report**
gemini-3.5-flash-lite	with batching lines	1 (merged everything into one query)	2	5 s	**✓ Report, but thinner**
gemini-3.8-flash	both	–	–	–	**✕ Not tested: 503 “high demand” on every try**

The winning run, step by step (gemini-3.6-flash, batching lines)



Measured, not illustrative. Each BigQuery job ran on its own thread. Gemini accepted LangChain's tool results without call ids on 3.6 Flash, which was an open question in the research.

What the live test showed

- **The batching lines matter.** On the same model they turned “limit hit, no report” into a full report in 3 calls. They've been in `prompts/system.md` since 2026-11-01.
- **Answer on the last call.** When the limit was hit, the user got nothing useful. Since then the last call has no tools and must answer from the results it has (`CallBudget`), and for a report it says the report can't be saved then.
- **Flash-Lite doesn't batch.** It merged everything into one query instead. It still works as a fallback model, but its reports are thinner.
- **3.6 Flash is slow per call** (16–20 s, because it thinks a lot). The time saved comes from BigQuery and from fewer rounds, not from faster LLM calls.
- **Gemini 3.8 Flash was overloaded all day** (503), so it's still untested. That's one more reason for the model fallback in step 4.

What was tested

WHAT	TESTED?	HOW
Gemini sends several SQL queries in one reply	**✓ Live, 3.6 Flash**	6 queries in the first reply, with the batching lines in the prompt. Without them: one query per reply
Real BigQuery runs them at the same time, as separate jobs	**✓ Live**	6 jobs started at 16.33 s on 6 threads and finished by 18.17 s
One failed query doesn't kill the others	**✓ Live**	q1 failed; the other 5 results were kept and only q1 was re-run (ticket 03)
Gemini accepts tool results without call ids	**✓ Live, 3.6 Flash**	LangChain 4.4.2 sends none; Google's 3.8 notes ask for them[7]. Still unchecked on 3.8
The model batches on Anthropic too	**✓ Live**	Claude Haiku with a fake `run_sql`: 4 queries in the first reply, 3 LLM calls
LangChain runs tool calls at the same time	**✓ Offline**	Fake model and tool: 3 calls taking 1.0 s, 0.2 s and 0.6 s finished in 1.01 s in total
gemini-3.8-flash (the default model)	**✕ Not yet**	503 “high demand” on every try on 2026-10-31
Free-tier limits for Gemini 3.8 Flash	**✕ Unknown**	No longer in Google's public docs; they only show in AI Studio[4]

References

1. Kim et al., *An LLM Compiler for Parallel Function Calling*, ICML 2024. arxiv.org/abs/2312.04511 Up to 3.7× faster, 6.7× cheaper, ~9% more accurate than ReAct; about the same accuracy as native parallel calls.
2. Yao et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023. arxiv.org/abs/2210.03629 The step-by-step loop; it can update its plan from each result.
3. Xu et al., *ReWOO: Decoupling Reasoning from Observations*, 2023. arxiv.org/abs/2305.18323 Planning all calls first: 5× fewer tokens and +4% accuracy on HotpotQA.
4. Google, *Gemini API rate limits* (updated 2026-10-28). ai.google.dev/gemini-api/docs/rate-limits Limits per minute on requests and input tokens; free-tier numbers only in AI Studio.
5. Pourreza & Rafiei, *DIN-SQL*, NeurIPS 2023. arxiv.org/abs/2304.11015 Nested queries need special handling.
6. Lei et al., *Spider 2.0*, ICLR 2025. arxiv.org/abs/2411.07763 Long, multi-step enterprise SQL: the best agent solved 21.3%.
7. Google, *Gemini function calling (generateContent)* (updated 2026-10-15), and *What's new in Gemini 3.8 Flash*. ai.google.dev/…/function-calling · ai.google.dev/…/whats-new-gemini-3.8 Parallel and compositional function calling on Flash models; the text-before-tool-call caveat; call ids.
8. Anthropic, *Parallel tool use*. platform.claude.com/…/parallel-tool-use Batch only independent, read-only calls; return all results together.
9. Google, *ADK: tool performance*. google.github.io/adk-docs/tools/performance Prompt for parallel calls; prefer several specific calls over one complex call.
10. Google Cloud, *BigQuery pricing*. cloud.google.com/bigquery/pricing $6.25 per TiB, first 1 TiB a month free, at least 10 MB per table referenced.
11. Google Cloud, *GoogleSQL query syntax: WITH clause* (2026-10-27). cloud.google.com/…/query-syntax A non-recursive CTE runs once for each place it's referenced.
12. Anthropic, *Building effective agents* (2024-12-19). anthropic.com/engineering/building-effective-agents Start simple; use parallelization for independent subtasks.

All figures and sources come from the research note (2026-10-31), which also lists the library source lines and the experiments behind them.